PharmaCare

Pharmacy Management System

Complete Technical Deep Dive

Every file, every line, every interaction explained

SQLite · Bootstrap 5 · Jinja2 · Python · Chart.js · QuaggaJS · openpyxl · REST API

Project structure · Code walkthrough · File interactions · Database schema · API reference

Table of Contents

1. Technology Stack — Every Library & Framework 3

2. Project Structure — Folder & File Map 4

3. Database Schema — Tables, Columns & Relationships 5

4. app.py — Complete Code Walkthrough (797 lines) 6

4.1. App Configuration & Constants 6

4.2. Database Helper Functions 7

4.3. Schema Bootstrap (init_db) 7

4.4. Authentication Decorators 8

4.5. Login / Logout Routes 8

4.6. Dashboard Route 9

4.7. Medicines CRUD Routes 10

4.8. Export to Excel Route 11

4.9. Alerts Route 12

4.10. Reports Route 12

4.11. Sales Route 13

4.12. Users Route 13

4.13. REST API Routes 14

4.14. Context Processor 15

5. Templates — All 12 HTML Files Explained 16

5.1. base.html — Master Layout 16

5.2. login.html 17

5.3. dashboard.html 17

5.4. medicines.html + _medicine_form.html 18

5.5. medicine_detail.html 19

5.6. alerts.html 19

5.7. sales.html 20

5.8. reports.html 20

5.9. barcode.html 21

5.10. users.html 22

5.11. logs.html + profile.html 22

6. style.css — Every CSS Rule Explained 23

7. main.js — JavaScript Logic 24

8. seed_data.py — Demo Data Seeding 24

9. How Files Interact — Complete Request Flow 25

10. How the Project Was Built — Step by Step 27

Section 1 — Technology Stack

Every library, framework and tool used in this project

Backend Technologies

Technology	Version	Role in Project	Why Chosen
Python	3.9+	Core language — all server-side logic lives here	Easy, readable, huge ecosystem, built-in SQLite
Flask	≥3.0.0	Micro web framework — routes, sessions, templates	Lightweight, no bloat, perfect for small-medium apps
Werkzeug	≥3.0.0	Password hashing (pbkdf2) + secure file uploads	Ships with Flask; handles security automatically
SQLite	Built-in	Relational database — stores all data in one file	Zero setup, file-based, ideal for single-server apps
openpyxl	≥3.1.2	Reads and writes .xlsx Excel files with formatting	Pure Python, supports cell colours and fonts
Jinja2	Ships w/Flask	HTML templating engine — renders dynamic pages	Syntax like Python, supports inheritance, loops, filters

Frontend Technologies

Technology	Version	Role in Project
Bootstrap 5	5.3.3	Responsive CSS grid, navbar, cards, modals, badges, tables, forms — all the UI components
Bootstrap Icons	1.11.3	500+ SVG icons — used throughout sidebar, buttons, stat cards (bi-capsule, bi-grid-fill, etc.)
Google Fonts (Inter)	—	Clean, modern sans-serif font loaded from Google CDN for the entire UI
Chart.js	4.4.0	JavaScript charting library — powers the 7-day sales line chart and stock donut chart on Dashboa
QuaggaJS	0.12.1	Open-source barcode scanner library — uses device camera to detect and decode barcodes in re

Python Standard Library Modules Used

Module	Usage in app.py
os	os.makedirs() to auto-create folders; os.path.join() to build cross-platform file paths
sqlite3	Direct database connection, cursor execution, row_factory for dict-like row access
datetime	date.today() for expiry comparisons; timedelta for +30 day warning window; strftime for formatting
functools.wraps	Preserves function metadata when creating decorator wrappers (login_required, roles_required)
io.BytesIO	In-memory byte buffer to serve the Excel file as a download without saving to disk first
json	json.dumps() to safely pass Python lists into Chart.js JavaScript arrays in templates

Section 2 — Project Structure

Every folder and file with its purpose

Complete Folder & File Map

```
pharmacy_app/
├── app.py                    ← Main application (797 lines) — ALL routes, DB logic, auth
├── seed_data.py              ← One-time demo data script (188 lines)
├── requirements.txt          ← Python dependencies (3 packages)
├── README.md                 ← Setup instructions and API reference
│
├── instance/                 ← Auto-created by Flask
│   └── pharmacy.db           ← SQLite database file (binary, ~50KB)
│
├── static/                   ← Files served directly to browser
│   ├── css/
│   │   └── style.css         ← Custom styles (188 lines)
│   ├── js/
│   │   └── main.js           ← Custom JavaScript (43 lines)
│   └── uploads/              ← Medicine images (auto-created)
│       └── <timestamp>_img.jpg
│
└── templates/                ← Jinja2 HTML templates
    ├── base.html             ← Master layout (204 lines) — all pages extend this
    ├── login.html            ← Standalone login page (99 lines)
    ├── dashboard.html        ← Stats + charts (267 lines)
    ├── medicines.html        ← Inventory list + search (247 lines)
    ├── _medicine_form.html   ← Reusable form partial (69 lines)
    ├── medicine_detail.html  ← Single medicine view (163 lines)
    ├── alerts.html           ← Expiry + low stock (151 lines)
    ├── sales.html            ← Sales management (121 lines)
    ├── reports.html          ← Analytics charts (175 lines)
    ├── barcode.html          ← Camera scanner (210 lines)
    ├── users.html            ← User management (116 lines)
    ├── logs.html             ← Activity audit (57 lines)
    └── profile.html          ← Account settings (50 lines)
```

File Size Summary

File	Lines	Size	Key Responsibility
app.py	797	~32 KB	All Flask routes, DB operations, auth, Excel export, REST API
base.html	204	~9 KB	Navbar with search+alerts, sidebar with role-based links, flash messages
dashboard.html	267	~11 KB	Stat cards, Chart.js charts, recent meds table, activity log
medicines.html	247	~10 KB	Search bar, paginated table, Add/Edit modals, row colour coding
medicine_detail.html	163	~7 KB	Full product view with image, clinical info, edit/delete modal
barcode.html	210	~9 KB	QuaggaJS camera init, live scan, async API fetch, history log

reports.html	175	~7 KB	Chart.js bar chart, 4 report tables, export button
style.css	188	~5 KB	CSS variables, navbar, sidebar, stat cards, tables, login, responsive
seed_data.py	188	~9 KB	30 medicines (expired/expiring/low/normal), 20 sales, 20 logs
main.js	43	~1.5 KB	Auto-dismiss alerts, sidebar highlight, confirm-delete, image preview

Section 3 — Database Schema

4 SQLite tables — structure, columns and relationships

How the database works

SQLite is a file-based database — no server needed. The entire database is a single file: instance/pharmacy.db. Python's built-in sqlite3 module connects to it directly. Flask uses WAL (Write-Ahead Logging) journal mode for better concurrent performance.

Table 1: users — Authentication & Role Management

Column	Type	Constraints	Purpose
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique user identifier, auto-increments
username	TEXT	UNIQUE NOT NULL	Login name — must be unique across all users
password	TEXT	NOT NULL	Werkzeug pbkdf2 password hash — never stored in plain text
role	TEXT	DEFAULT 'staff'	Access level: 'admin', 'pharmacist', or 'staff'
full_name	TEXT	—	Display name shown in navbar and profile
email	TEXT	—	Email address for the user (optional)
created_at	TEXT	DEFAULT datetime('now')	Timestamp auto-set when user is created

Table 2: medicines — Core Inventory

Column	Type	Purpose
id	INTEGER PK	Auto-increment primary key
ref_no	TEXT	Reference number (e.g. REF-001) — for procurement tracking
company_name	TEXT	Manufacturer name (e.g. Sun Pharma, Cipla)
medicine_type	TEXT	Category: Tablet, Capsule, Syrup, Injection, Cream, Drops, Inhaler, Sachet
medicine_name	TEXT NOT NULL	Required — full product name including strength (e.g. Paracetamol 500mg)
lot_no	TEXT	Batch/lot number for traceability
issue_date	TEXT	Manufacturing/issue date stored as ISO string YYYY-MM-DD
exp_date	TEXT	Expiry date as ISO string — compared with date.today() for alerts
uses	TEXT	Medical indications / what the medicine treats
side_effect	TEXT	Known adverse effects
precautions	TEXT	Warnings — drug interactions, contraindications
dosage	TEXT	Recommended dosage instructions
price	REAL	Unit price — used for sales total calculation and inventory value
product_qty	INTEGER DEFAULT 0	Current stock count — decremented on each sale

image_filename	TEXT	Stored filename of uploaded medicine photo (in static/uploads/)
barcode	TEXT	Barcode number — used by the scanner to look up medicine
created_at	TEXT	Auto timestamp when record was created
updated_at	TEXT	Auto-updated to datetime('now') on every edit

Table 3: sales — Transaction Records

Column	Type	Purpose
id	INTEGER PK	Auto-increment sale ID
medicine_id	INTEGER	FK → medicines.id (soft link, not enforced at DB level)
medicine_name	TEXT	Denormalized name — keeps record even if medicine deleted
quantity	INTEGER	Units sold in this transaction
unit_price	REAL	Price at time of sale — snapshot, not live reference
total_price	REAL	Pre-calculated: unit_price × quantity (rounded to 2 decimal places)
sold_by	TEXT	Username of the staff member who recorded the sale
sale_date	TEXT	Datetime of sale — used for 7-day chart grouping on dashboard

Table 4: activity_logs — Audit Trail

Column	Type	Purpose
id	INTEGER PK	Auto-increment log ID
user_id	INTEGER	ID of user who performed the action
username	TEXT	Username snapshot (preserved even if user deleted)
action	TEXT	Action code: LOGIN, LOGOUT, ADD_MEDICINE, UPDATE_MEDICINE, DELETE_MEDICINE, SALE, E
details	TEXT	Human-readable description, e.g. 'Sold 5x Paracetamol for $27.50'
ip_address	TEXT	Client IP from request.remote_addr — for security auditing
created_at	TEXT	Timestamp of when the action occurred

Section 4 — app.py — Complete Code Walkthrough

797 lines explained section by section

4.1 App Configuration & Constants (Lines 1–28)

```
import os, sqlite3, json, io
from datetime import datetime, timedelta, date
from functools import wraps
from flask import Flask, render_template, request, redirect,
                  url_for, session, flash, jsonify, send_file, g
from werkzeug.security import generate_password_hash, check_password_hash
from werkzeug.utils import secure_filename
import openpyxl
from openpyxl.styles import Font, PatternFill, Alignment

app = Flask(__name__)
app.secret_key = 'pharmacy_secret_key_2024'
app.config['UPLOAD_FOLDER'] = os.path.join('static', 'uploads')
app.config['MAX_CONTENT_LENGTH'] = 5 * 1024 * 1024   # 5 MB limit
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif', 'webp'}
LOW_STOCK_THRESHOLD = 10          # qty ≤ 10 triggers alert
EXPIRY_WARN_DAYS = 30             # days ahead to warn about expiry
DATABASE = os.path.join('instance', 'pharmacy.db')
os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)
os.makedirs('instance', exist_ok=True)
```

Code	What It Does
Flask(__name__)	Creates the Flask app object. __name__ = module name, helps Flask find templates/static
app.secret_key	32-byte string used to sign session cookies — changing it logs out all users
UPLOAD_FOLDER	Path where medicine images are saved: static/uploads/ inside the app directory
MAX_CONTENT_LENGTH = 5MB	Flask rejects any uploaded file larger than 5MB with HTTP 413 automatically
LOW_STOCK_THRESHOLD = 10	Change this number in one place to change alert threshold for entire app
EXPIRY_WARN_DAYS = 30	Medicines expiring within 30 days show in Alerts. Change here to adjust window
os.makedirs(..., exist_ok=True)	Creates the folders if they don't exist yet — safe to run repeatedly

4.2 Database Helper Functions (Lines 30–60)

```python
def get_db():
    db = getattr(g, '_database', None)
    if db is None:
        db = g._database = sqlite3.connect(DATABASE)
        db.row_factory = sqlite3.Row
        db.execute('PRAGMA journal_mode=WAL')
    return db

@app.teardown_appcontext
def close_connection(exception):
    db = getattr(g, '_database', None)
    if db is not None: db.close()

def query_db(sql, args=(), one=False, commit=False):
    db = get_db()
    cur = db.execute(sql, args)
    if commit:
        db.commit()
        return cur.lastrowid
    rv = cur.fetchall()
    return (rv[0] if rv else None) if one else rv
```

Code	Explanation
g (Flask global)	Flask's 'g' object lives for exactly ONE request. It's the right place to store a DB connection.
getattr(g, '_database', None)	Retrieves existing connection from g if it exists, else returns None (avoids AttributeError)
db.row_factory = sqlite3.Row	Makes rows accessible as dict-like objects: row['column_name'] instead of row[0]
PRAGMA journal_mode=WAL	Write-Ahead Logging: readers don't block writers. Improves performance with concurrent access.
@app.teardown_appcontext	Runs after every request. Closes the DB connection to prevent resource leaks.
query_db(sql, args, one, commit)	Universal DB utility. args= prevents SQL injection. one=True returns single row. commit=True saves char
cur.lastrowid	After INSERT, returns the auto-generated ID of the new row — useful to redirect to new record

4.3 Schema Bootstrap — init_db() (Lines 62–110)

```python
def init_db():
    db = sqlite3.connect(DATABASE)
    c = db.cursor()
    c.executescript('''
    CREATE TABLE IF NOT EXISTS users (...);
    CREATE TABLE IF NOT EXISTS medicines (...);
    CREATE TABLE IF NOT EXISTS activity_logs (...);
    CREATE TABLE IF NOT EXISTS sales (...);
    ''')
    db.commit()
    # Seed admin if none exists
    row = db.execute('SELECT id FROM users WHERE role=...')
    if not row:
        pw = generate_password_hash('admin123')
        db.execute('INSERT INTO users ...')
        db.commit()

if __name__ == '__main__':
    init_db()
    app.run(debug=True, use_reloader=False)
```

Idempotent schema creation

CREATE TABLE IF NOT EXISTS means init_db() is safe to run multiple times — it won't destroy existing data or raise errors if tables already exist. This is called only when running app.py directly (the if __name__ == '__main__' guard).

Why use_reloader=False

use_reloader=False prevents Flask from starting two processes in debug mode, which would cause init_db() to run twice and potentially cause a locking conflict on the SQLite file.

4.4 Authentication Decorators (Lines 112–140)

```
def login_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if 'user_id' not in session:
            flash('Please login first.', 'warning')
            return redirect(url_for('login'))
        return f(*args, **kwargs)
    return decorated

def roles_required(*roles):
    def decorator(f):
        @wraps(f)
        def decorated(*args, **kwargs):
            if session.get('role') not in roles:
                flash('Access denied.', 'danger')
                return redirect(url_for('dashboard'))
            return f(*args, **kwargs)
        return decorated
    return decorator
```

Concept	Explanation
Decorator pattern	A function that wraps another function to add behaviour before/after it runs — Python's @syntax sugar
@wraps(f)	Copies __name__, __doc__ from the original function to the wrapper. Without it, Flask's URL routing breaks
session['user_id']	Flask's signed cookie. Contains user_id, username, role, full_name after login. Cleared on logout.
*roles parameter	Allows @roles_required('admin','pharmacist') — accepts any number of allowed roles
Usage on routes	@app.route('/add_medicine') + @login_required + @roles_required('admin','pharmacist') stack from top to b

4.5 Login & Logout Routes (Lines 165–195)

```python
@app.route('/login', methods=['GET', 'POST'])
def login():
    if 'user_id' in session:              # already logged in?
        return redirect(url_for('dashboard'))
    if request.method == 'POST':
        uname = request.form.get('username','').strip()
        pw    = request.form.get('password', '')
        user  = query_db('SELECT * FROM users WHERE username=?', (uname,), one=True)
        if user and check_password_hash(user['password'], pw):
            session['user_id']   = user['id']
            session['username']  = user['username']
            session['role']      = user['role']
            session['full_name'] = user['full_name']
            log_activity('LOGIN', f'User logged in from {request.remote_addr}')
            return redirect(url_for('dashboard'))
        flash('Invalid username or password.', 'danger')
    return render_template('login.html')

@app.route('/logout')
@login_required
def logout():
    log_activity('LOGOUT', 'User logged out')
    session.clear()              # removes all session data
    return redirect(url_for('login'))
```

Code	What Happens
methods=['GET','POST']	GET = show empty form; POST = process submitted form data
query_db(..., one=True)	Fetches exactly one user row or None if not found
check_password_hash(stored, input)	Werkzeug compares the stored pbkdf2 hash against the typed password. Never compares pla
session[...] = user[...]	Flask writes these key-value pairs into a signed cookie on the browser
session.clear()	Removes all keys from the session cookie — effectively logs the user out
log_activity(...)	Inserts a row into activity_logs — called after every significant action

4.6 Dashboard Route (Lines 197–265)

```python
@app.route('/')
@login_required
def dashboard():
    total_medicines = query_db('SELECT COUNT(*) as c FROM medicines', one=True)['c']
    total_qty       = query_db('SELECT COALESCE(SUM(product_qty),0)...')['s']
    total_value     = query_db('SELECT COALESCE(SUM(price*product_qty),0)...')['v']
    total_sales     = query_db('SELECT COALESCE(SUM(total_price),0)...')['s']
    today     = date.today()
    warn_date = today + timedelta(days=EXPIRY_WARN_DAYS)   # 30 days ahead
    expiring_count  = query_db('...WHERE exp_date <= ?', (warn_date.isoformat(),))
    expired_count   = query_db('...WHERE exp_date < ?', (today.isoformat(),))
    low_stock_count = query_db('...WHERE product_qty <= ?', (LOW_STOCK_THRESHOLD,))
    # Sales chart — 7 days
    chart_labels, chart_data = [], []
    for i in range(6, -1, -1):       # 6 days ago → today
        d = (today - timedelta(days=i)).isoformat()
        row = query_db('SELECT SUM(total_price) FROM sales WHERE date(sale_date)=?', (d,))
        chart_labels.append((today-timedelta(days=i)).strftime('%b %d'))
        chart_data.append(float(row['s']))
    return render_template('dashboard.html',
        chart_labels=json.dumps(chart_labels),
        chart_data=json.dumps(chart_data), ...)
```

COALESCE pattern

COALESCE(SUM(col), 0) handles the case where the table is empty — SUM of nothing returns NULL in SQL, and COALESCE replaces NULL with 0 so the Python code never crashes. The dashboard runs 10+ separate SQL queries on every page load.

Passing data to Chart.js

json.dumps() converts Python lists like ['Mar 10', 'Mar 11', ...] into valid JSON strings that are injected directly into JavaScript in the template: const labels = {{ chart_labels|safe }}. The |safe filter tells Jinja2 not to HTML-escape the brackets.

4.7 Medicines CRUD Routes (Lines 267–390)

CRUD = Create, Read, Update, Delete — the four fundamental database operations.

```
@app.route('/medicines')
@login_required
def medicines():
    search = request.args.get('search', '').strip()
    page   = max(1, int(request.args.get('page', 1)))
    per    = 10                          # items per page
    offset = (page - 1) * per            # SQL OFFSET
    base_sql = 'FROM medicines'
    args = []
    if search:
        like = f'%{search}%'             # % = wildcard in SQL LIKE
        where = ' WHERE medicine_name LIKE ? OR company_name LIKE ? ...'
        base_sql += where
        args = [like, like, like, like, like]
    total = query_db('SELECT COUNT(*) as c ' + base_sql, args, one=True)['c']
    meds  = query_db('SELECT * ' + base_sql + ' LIMIT ? OFFSET ?', args+[per,offset])
    pages = (total + per - 1) // per     # ceiling division

@app.route('/add_medicine', methods=['POST'])
@roles_required('admin', 'pharmacist')
def add_medicine():
    d = request.form                     # ImmutableMultiDict of form fields
    file = request.files.get('image')    # uploaded file object
    if file and allowed_file(file.filename):
        filename = secure_filename(f'{datetime.now():%Y%m%d%H%M%S}_{file.filename}')
        file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
    query_db('INSERT INTO medicines ...', (...), commit=True)
    log_activity('ADD_MEDICINE', f"Added: {d.get('name')}")
    flash(f"Medicine added!", 'success')
    return redirect(url_for('medicines'))
```

Pattern	Explanation
request.args.get('search','')	Query string params (?search=para). Empty string default prevents None errors.
LIMIT ? OFFSET ?	Pagination SQL. Page 1: LIMIT 10 OFFSET 0. Page 2: LIMIT 10 OFFSET 10. Page 3: OFFSET 20.
(page-1)*per	Offset formula. Page 1 → 0, Page 2 → 10, Page 3 → 20. max(1, ...) prevents page 0 or negative.
(total+per-1)//per	Ceiling division for total pages. 11 items ÷ 10 = 1.1 → ceil = 2 pages. Python integer trick.
LIKE '%search%'	SQL wildcard search. % matches zero or more characters. Case-insensitive in SQLite by default.
secure_filename()	Strips dangerous characters from filenames: ../../../etc/passwd becomes etcpasswd
{datetime.now():%Y%m%d%H%M%S}_	Prefixes timestamp to prevent filename collisions when two users upload same filename
redirect(url_for('medicines'))	After POST, always redirect (PRG pattern). Prevents form resubmission on browser refresh.

4.8 Export to Excel Route (Lines 392–440)

```python
@app.route('/export_excel')
@login_required
def export_excel():
    meds = query_db('SELECT * FROM medicines ORDER BY medicine_name')
    wb = openpyxl.Workbook()            # create new Excel workbook
    ws = wb.active                      # get first (default) sheet
    ws.title = 'Medicines'
    header_fill = PatternFill('solid', fgColor='1a6b3c')  # green fill
    header_font = Font(bold=True, color='FFFFFF')          # white bold text
    # Write headers in row 1
    for col, h in enumerate(headers, 1):
        cell = ws.cell(row=1, column=col, value=h)
        cell.font = header_font
        cell.fill = header_fill
    # Write data rows, colour-code expired/low stock
    for row, m in enumerate(meds, 2):
        if exp < today:                 # expired → red row
            fill = PatternFill('solid', fgColor='FFCCCC')
        elif qty <= LOW_STOCK_THRESHOLD:# low stock → yellow row
            fill = PatternFill('solid', fgColor='FFF3CD')
    buf = io.BytesIO()                  # in-memory file buffer
    wb.save(buf)                        # save Excel to buffer, not disk
    buf.seek(0)                         # rewind to beginning
    return send_file(buf,
        download_name=f'medicines_{date.today()}.xlsx',
        as_attachment=True,
        mimetype='application/vnd.openxmlformats...')
```

In-memory file serving

io.BytesIO() is an in-memory binary stream — the Excel file is generated in RAM and streamed directly to the browser. No temporary file is ever written to disk, which is cleaner and faster.

4.9 Alerts Route (Lines 442–460)

```
def get_alerts():
    today = date.today()
    warn_date = today + timedelta(days=EXPIRY_WARN_DAYS)
    expiring = query_db(                # includes both expired AND expiring soon
        'SELECT * FROM medicines WHERE exp_date IS NOT NULL'
        ' AND exp_date != "" AND exp_date <= ? ORDER BY exp_date',
        (warn_date.isoformat(),))
    low_stock = query_db(
        'SELECT * FROM medicines WHERE product_qty <= ? ORDER BY product_qty',
        (LOW_STOCK_THRESHOLD,))
    expired = [m for m in expiring if m['exp_date'] <= today.isoformat()]
    return expiring, low_stock, expired   # expiring includes expired subset

@app.route('/alerts')
@login_required
def alerts():
    expiring, low_stock, expired = get_alerts()
    return render_template('alerts.html', expiring=expiring,
        low_stock=low_stock, expired=expired, ...)
```

Shared logic pattern

get_alerts() is a shared helper used by BOTH the /alerts route AND the context_processor. This means the alerts count in the navbar badge comes from the exact same logic as the Alerts page — no inconsistency possible.

4.10 Reports Route (Lines 462–510)

```
@app.route('/reports')
@roles_required('admin', 'pharmacist')
def reports():
    by_type = query_db('''
        SELECT medicine_type,
               COUNT(*) as count,
               COALESCE(SUM(product_qty),0) as total_qty,
               COALESCE(SUM(price*product_qty),0) as total_value
        FROM medicines
        GROUP BY medicine_type
        ORDER BY count DESC
    ''')
    # Top 10 lowest stock medicines
    low_stock_report = query_db(
        'SELECT * FROM medicines ORDER BY product_qty ASC LIMIT 10')
    # 30-day sales summary grouped by day
    sales_summary = query_db('''
        SELECT date(sale_date) as day, COUNT(*) as transactions,
               SUM(quantity) as units, SUM(total_price) as revenue
        FROM sales GROUP BY day ORDER BY day DESC LIMIT 30
    ''')
    type_labels = json.dumps([r['medicine_type'] or 'Unknown' for r in by_type])
    type_data   = json.dumps([r['total_qty'] for r in by_type])
```

4.11 Sales Routes (Lines 512–560)

```
@app.route('/add_sale', methods=['POST'])
@roles_required('admin', 'pharmacist', 'staff')
def add_sale():
    med_id = request.form.get('medicine_id')
    qty    = int(request.form.get('quantity', 1))
    med    = query_db('SELECT * FROM medicines WHERE id=?', (med_id,), one=True)
    if med['product_qty'] < qty:          # insufficient stock check
        flash(f'Insufficient stock. Available: {med["product_qty"]}', 'warning')
        return redirect(url_for('sales'))
    total = round(med['price'] * qty, 2)  # avoid floating point drift
    query_db('INSERT INTO sales ...', commit=True)
    # Atomically reduce stock by qty sold
    query_db('UPDATE medicines SET product_qty=product_qty-? WHERE id=?',
             (qty, med_id), commit=True)
    log_activity('SALE', f'Sold {qty}x {med["medicine_name"]} for ${total}')
```

Atomic stock deduction

product_qty = product_qty - qty uses SQL arithmetic directly in the UPDATE statement. This is safer than: fetch qty, subtract in Python, then update — because another concurrent request could modify qty between fetch and update (race condition). SQL does it atomically.

4.12 User Management Routes (Lines 562–620)

```
@app.route('/add_user', methods=['POST'])
@roles_required('admin')
def add_user():
    existing = query_db('SELECT id FROM users WHERE username=?', (uname,), one=True)
    if existing:
        flash('Username already exists.', 'danger')
        return redirect(url_for('users'))
    query_db('INSERT INTO users (username,password,role,...) VALUES (?,?,?,?,?)',
             (uname, generate_password_hash(pw), role, fname, email), commit=True)

@app.route('/delete_user/<int:id>')
@roles_required('admin')
def delete_user(id):
    if id == session['user_id']:          # prevent self-deletion
        flash('Cannot delete your own account.', 'danger')
        return redirect(url_for('users'))
```

4.13 REST API Routes (Lines 622–740)

```
def api_auth(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        api_key = request.headers.get('X-API-Key')
        token   = request.headers.get('Authorization','').replace('Bearer ','')
        if 'user_id' in session: return f(*args, **kwargs)  # web session OK
        if api_key == app.secret_key or token == app.secret_key:
            return f(*args, **kwargs)                       # API key OK
        return jsonify({'error':'Unauthorized'}), 401
    return decorated

@app.route('/api/medicines', methods=['GET'])
@api_auth
def api_get_medicines():
    search = request.args.get('search','')
    if search:
        meds = query_db('...WHERE medicine_name LIKE ? OR ...')
    return jsonify([dict(m) for m in meds])   # Row → dict → JSON

@app.route('/api/add_medicine', methods=['POST'])
@api_auth
def api_add_medicine():
    d = request.get_json()              # parses JSON body
    if not d or not d.get('medicine_name'):
        return jsonify({'error':'medicine_name required'}), 400
    lid = query_db('INSERT INTO medicines ...', commit=True)
    return jsonify({'success': True, 'id': lid}), 201
```

Endpoint	Method	Auth	Returns
/api/medicines	GET	Session or X-API-Key header	JSON array of all medicines

/api/medicines/<id>	GET	Session or X-API-Key	Single medicine JSON or 404
/api/add_medicine	POST	Session or X-API-Key	{'success':True,'id':N} or 400
/api/update_medicine/<id>	PUT	Session or X-API-Key	{'success':True} or 404
/api/delete_medicine/<id>	DELETE	Session or X-API-Key	{'success':True} or 404
/api/stats	GET	Session or X-API-Key	Dashboard counts as JSON
/api/barcode/<code>	GET	Session or X-API-Key	Medicine matching barcode or 404

4.14 Context Processor (Lines 742–760)

```
@app.context_processor
def inject_alerts():
    if 'user_id' in session:              # only for logged-in users
        today     = date.today()
        warn_date = today + timedelta(days=EXPIRY_WARN_DAYS)
        exp_count = query_db(              # count expiring medicines
            'SELECT COUNT(*) FROM medicines WHERE exp_date <= ?',
            (warn_date.isoformat(),), one=True)['c']
        ls_count  = query_db(              # count low stock medicines
            'SELECT COUNT(*) FROM medicines WHERE product_qty <= ?',
            (LOW_STOCK_THRESHOLD,), one=True)['c']
        return dict(nav_expiring=exp_count, nav_low_stock=ls_count)
    return dict(nav_expiring=0, nav_low_stock=0)
```

Context Processor — runs on every page

@app.context_processor injects variables into EVERY template automatically — no need to pass them in every render_template() call. nav_expiring and nav_low_stock are how base.html shows the red badge number on the Alerts nav link and the notification bell. This runs on EVERY page load for logged-in users, executing 2 SQL queries each time.

Section 5 — Templates — All 12 HTML Files

Jinja2 templating, inheritance, and page-by-page explanation

How Jinja2 inheritance works

Jinja2 Template Inheritance: Every page (except login.html) starts with {% extends 'base.html' %}. This means base.html renders first — providing the navbar, sidebar, flash messages, JS includes — then the child page fills in {% block content %}...{% endblock %} with its specific content.

5.1 base.html — The Master Layout (204 lines)

Section in base.html	What It Provides to Every Page
<head> CDN links	Loads Bootstrap 5 CSS, Bootstrap Icons, Google Inter font, custom style.css
{% block head %}	Child templates can inject extra <link> or <meta> tags (barcode.html uses this for QuaggaJS)
#mainNavbar	Fixed green top bar with logo, global search form (→ medicines?search=), bell icon with alert badge, u
Bell notification dropdown	Shows nav_expiring and nav_low_stock counts injected by context_processor — links to /alerts
User dropdown	Shows session.full_name initial in avatar, role badge, links to Profile/Users/Logs/Logout
#sidebar	Left sidebar with role-conditional links. Admin sees Users+Logs. Pharmacist sees Reports+Export. Sta
request.endpoint check	Adds CSS class 'active' to the current page's sidebar link: {% if request.endpoint=='dashboard' %}activ
Flash messages block	{% with messages = get_flashed_messages(with_categories=true) %} — renders Bootstrap alerts for s
{% block content %}	Empty placeholder filled by every child template with its own page content
Bootstrap bundle JS	Enables Bootstrap modals, dropdowns, tooltips, collapse (navbar burger on mobile)
Chart.js CDN	Makes Chart global available to dashboard.html and reports.html {% block scripts %}
main.js	Auto-dismiss success/info alerts after 4s, sidebar active link, confirm-delete dialogs, image preview
{% block scripts %}	Child templates inject page-specific JavaScript here (Chart init, Quagga init, barcode lookup)

5.2 login.html — Standalone Page (99 lines)

Feature	Code / Explanation
Does NOT extend base.html	Login has its own <html>...<body> with class='login-page' — full green gradient background, no navbar/s
togglePassword() JS	onclick swaps input type between 'password' and 'text', changes eye icon between bi-eye and bi-eye-slas
Demo credentials panel	Hard-coded admin/pharmacist/staff credentials shown in the UI for development convenience
Flash messages	Inline {% with messages %} block shows login failure errors (Invalid username or password)
POST action	<form method='post' action='/login'> — submits username+password to the /login route
autofocus	Input autofocus attribute so username field is focused immediately on page load — no click needed

5.3 dashboard.html (267 lines)

Component	How It Works
4 stat cards	Bootstrap col-6 col-lg-3 grid. Displays total_medicines, total_qty, total_value, total_sales from route
Alert banners	{% if expired_count > 0 %} shows Bootstrap alert-danger. alert-warning for expiring. alert-info for low stoc
Sales line chart	<canvas id='salesChart'>. In {% block scripts %}: new Chart(...) with labels={{ chart_labels|safe }} — Pyth
Stock donut chart	<canvas id='stockChart'>. Data: [total_medicines-low_stock_count, low_stock_count, expiring_count]
Recent medicines table	{% for m in recent_meds %} loops last 5 medicines. Qty badge is bg-danger if ≤10, bg-success otherwise
Status logic in template	{% if m['exp_date'] < today %} Expired {% elif m['exp_date'] <= warn_date %} Expiring {% else %} OK{%
Activity log list	{% for log in recent_logs %} with dot colour: bg-danger for DELETE, bg-success for ADD/SALE, bg-warn

5.4 medicines.html + _medicine_form.html (247 + 69 lines)

Feature	Technical Detail
Search form	GET form → /medicines?search=... Uses request.args in route. 'Clear' button links back to /medicines wit
Table row colours	CSS classes: table-danger (exp<today), table-warning (exp≤warn_date OR qty≤threshold)
Pagination	Bootstrap .pagination with {% for p in range(1, pages+1) %}. Active page gets .active class.
Add Medicine modal	Button data-bs-toggle='modal' data-bs-target='#addMedModal'. Form POSTs to /add_medicine.
{% include '_medicine_form.html' %}	Inserts the 69-line reusable form partial into the modal body — same form used in both Add and Edit mod
Edit modal per row	Each table row has a unique modal ID: #editModal{{ m['id'] }}. The edit form POSTs to /update_medicine/
Delete link	href='/delete_medicine/{{ m.id }}' with onclick='return confirm(...)' — uses main.js data-confirm attribute
_medicine_form.html	16 fields: name(required), type, company, ref_no, lot, issue(date), exp(date), price(number), qty(number)

5.5 medicine_detail.html (163 lines)

Component	Detail
Breadcrumb nav	Medicines → Medicine Name — shows context, links back to list
Left column (col-lg-4)	Image (or placeholder capsule icon), name, type, status badges (Expired/Expiring/Valid + Low Stock), price
Right column (col-lg-8)	Two cards: Product Information (ref, company, lot, issue date, expiry date, dosage) and Clinical Information
Edit button	Only visible if session.role in ['admin','pharmacist']. Opens #editModal with all fields pre-filled with current va
Delete button	Only visible if session.role == 'admin'. Has confirm dialog. Links to /delete_medicine/{{ med.id }}.
Edit modal	Full medicine edit form with all fields pre-filled using value='{{ med[field] or "" }}' pattern

5.6 alerts.html (151 lines)

Section	Content
Summary cards row	3 cards: expired|length count in red, expiring|length in yellow, low_stock|length in blue
Expired section	{% if expired %} — red card header. Table shows: name, company, lot_no, exp_date badge in red, qty, View
Expiring Soon section	{% if expiring %} — yellow header. Extra column: Days Left = (exp_date - today) in Python/Jinja. Sorted by
Low Stock section	{% if low_stock %} — blue header. Shows qty badge, progress bar at qty*10% width (visual indicator), View
Jinja date math	{{ ((med.exp_date|strptime) - today).days }} — NOT used here; days left is passed from route as computed

5.7 sales.html (121 lines)

Feature	Technical Detail
New Sale modal	Button → #addSaleModal. Form POSTs to /add_sale.
Medicine dropdown	{% for m in medicines %} — route passes all medicines sorted by name. Shows: name — Stock: qty | Pri
Sales table	Paginated (15/page). Columns: id, medicine_name, quantity badge, unit_price, total_price (green), sold_

sale_date[:16]	Python/Jinja string slice: '2025-03-16 14:30:00' → '2025-03-16 14:30' — drops seconds for cleaner displa

5.8 reports.html (175 lines)

Component	Detail
4 summary stat cards	by_type|length (categories), total_inventory_value (formatted as $), low_stock_report|length, expiring_rep
Bar chart (typeChart)	Chart.js bar type. Labels from type_labels JSON. Data from type_data JSON. Colours array auto-genera
Inventory by Category table	Loops by_type: medicine_type, count badge, total_qty, total_value formatted with %.2f
Top 10 Low Stock table	ORDER BY product_qty ASC LIMIT 10. Progress bar: width={{ [m.product_qty*10, 100]|min }}% — Jinja2
Expiring Soon table	Same query as alerts page but displayed as report. Shows days left calculated in template.
30-Day Sales table	date(sale_date) groups by calendar day. Shows transactions count, units sold, revenue per day.

5.9 barcode.html (210 lines) — The Most Complex Template

> **JavaScript-heavy page**
> This is the most JavaScript-heavy page. It uses QuaggaJS (camera barcode detection), the Fetch API (async HTTP to /api/barcode/), browser camera permissions, and a local scan history stored in a JS array.

```
# In {% block head %} — loads QuaggaJS from CDN:
<script src='https://cdn.jsdelivr.net/npm/quagga@0.12.1/...'></script>

function startScanner() {
  Quagga.init({
    inputStream: { type: 'LiveStream',
      target: document.querySelector('#interactive'),  // DOM element
      constraints: { facingMode: 'environment' }      // rear camera
    },
    decoder: { readers: ['code_128_reader','ean_reader','upc_reader', ...] }
  }, function(err) {
    if (err) { alert('Camera error: ' + err); return; }
    Quagga.start();                    // starts video stream
  });
  Quagga.onDetected(function(result) {
    const code = result.codeResult.code;  // extracted barcode string
    stopScanner();
    lookupBarcode(code);              // call our API
  });
}

async function lookupBarcode(code) {
  const res = await fetch(`/api/barcode/${encodeURIComponent(code)}`);
  if (res.ok) {
    const med = await res.json();     // parse JSON response
    showResult(med);                  // render medicine card
  } else {
    document.getElementById('notFoundCard').classList.remove('d-none');
  }
}
```

JS Concept	Explanation
Quagga.init()	Initialises the barcode scanner library with camera stream and decoder configuration
facingMode: 'environment'	Requests the rear camera (back-facing) on mobile devices — better for scanning
readers array	Supported barcode formats: Code128 (most common), EAN-13 (grocery), EAN-8, Code39, UPC-A
Quagga.onDetected()	Callback fires every time a barcode is successfully decoded — gets the result object
async/await fetch()	Modern JavaScript async HTTP. No page reload needed — result shows instantly.
encodeURIComponent()	URL-encodes the barcode string — prevents special characters from breaking the URL
classList.remove('d-none')	Bootstrap utility class. d-none = display:none. Removing it shows the hidden card.
Tab system	Camera Scan tab (#cameraTab) and Manual Entry tab (#manualTab) — Bootstrap nav-tabs component

Scan history array	Local JS array (max 10 entries) — shows recent scans with code, found/not-found badge, time

5.10 users.html (116 lines)

Feature	Detail
Role summary cards	Jinja2 loop: {% for role, color, icon in [('admin','danger','shield-fill'), ...] %} — elegant data-driven card genera
users|selectattr filter	{{ users|selectattr('role','equalto',role)|list|length }} — Jinja2's built-in attribute filter counts users per role
User table	Avatar initials (first letter of name), username in <code>, email, role badge (role-admin/pharmacist/staff CSS
You badge	if u['id'] == session.user_id: shows 'You' badge instead of delete button — prevents self-deletion
Add User modal	Fields: full_name, username(required), email, password(required), role select(staff/pharmacist/admin)

5.11 logs.html (57 lines) + profile.html (50 lines)

File	Key Features
logs.html	Paginated (20/page). Action badge colour: bg-danger=DELETE, bg-success=ADD/SALE, bg-warning=UPDATE/EXPO
profile.html	Avatar-xl shows first letter of name. Three sections: display info, edit form (name+email), change password (requires c

Section 6 — style.css — Every CSS Rule Explained

188 lines of custom styles layered on top of Bootstrap 5

CSS Section	Lines	What It Styles
:root variables	1–6	--sidebar-width:240px, --navbar-height:58px, --primary:#1a6b3c (pharmacy green), --primary-dark
body	10–14	Inter font, #f0f4f8 light blue-grey background, #212529 dark text — overrides Bootstrap defaults
#mainNavbar	17–23	linear-gradient(135deg, green→#2e8b57) background, 58px height, z-index:1040 (above content), bo
.brand-icon	25–30	34×34px white-on-green rounded square containing the pill icon in top-left of navbar
.navbar-search	32–37	Max 420px search bar, rounded 8px input group, transparent/white layered background
.avatar-sm	39–44	32×32px circular avatar for user initials — shown in navbar user dropdown
#wrapper	47	padding-top:58px (pushes content below fixed navbar), min-height:100vh
#sidebar	49–58	240px wide, white bg, sticky positioned (stays when scrolling), shadow, overflow-y:auto for long menu
#sidebar .nav-link	60–65	Flex row with 10px gap, 8px rounded hover, color transitions. Active = green-tinted background.
.sidebar-divider	67–69	Tiny uppercase grey section label like 'ADMIN' and 'ACCOUNT' separating link groups
#page-content	72–76	flex:1 takes remaining width, 24px padding, overflow-x:hidden prevents horizontal scroll
.page-header	80–84	Bottom border separates page title from content, 1.5rem bottom margin
.stat-card	88–98	Dashboard stat card styles: 48×48px coloured icon square (12px radius), large 1.75rem value, 0.8rem
.med-thumb	101–105	36×36px grey pill icon placeholder when no medicine image uploaded
.log-dot	112–113	8×8px coloured circle in activity log timeline — fills with bg-danger/success/warning/info
.role-badge	115–118	Pill badges for admin(red), pharmacist(blue), staff(green) with subtle background and matching borde
.avatar-xl	120–125	80×80px green gradient circle for profile page — shows large initial letter
.login-page	128	Full-page green gradient background applied to body on login page only
.login-card	131–132	Max 440px centered card, 16px border radius — floated center on gradient background
.table th	136	0.8rem font, uppercase, 0.5px letter-spacing — Bootstrap table header enhancement
.card	139	`border-radius: 12px !important` — overrides Bootstrap's default 0.375rem for rounder cards everywh
.modal-content	143–144	16px border-radius, no border — rounder than Bootstrap default for modals
@media (max-width:768px)	148–155	Mobile: sidebar hidden (display:none), padding reduced, stat values smaller, search narrower

Section 7 — main.js — JavaScript Logic

43 lines that enhance every page

```
document.addEventListener('DOMContentLoaded', function() {
  // 1. Auto-dismiss success/info alerts after 4 seconds
  setTimeout(() => {
    document.querySelectorAll('.alert.alert-success, .alert.alert-info')
      .forEach(el => bootstrap.Alert.getOrCreateInstance(el).close());
  }, 4000);

  // 2. Highlight active sidebar link based on current URL
  const path = window.location.pathname;
  document.querySelectorAll('#sidebar .nav-link').forEach(link => {
    if (link.getAttribute('href') === path) link.classList.add('active');
  });

  // 3. Confirm-delete dialogs for any element with data-confirm attribute
  document.querySelectorAll('[data-confirm]').forEach(el => {
    el.addEventListener('click', function(e) {
      if (!confirm(this.dataset.confirm)) e.preventDefault();
    });
  });

  // 4. Bootstrap tooltip initialisation
  document.querySelectorAll('[data-bs-toggle="tooltip"]')
    .forEach(el => new bootstrap.Tooltip(el));

  // 5. Live image preview when file selected
  document.querySelectorAll('input[type="file"][accept*="image"]')
    .forEach(input => {
      input.addEventListener('change', function() {
        const file = this.files[0];
        if (file) {
          const reader = new FileReader();
          reader.onload = e => {
            preview.src = e.target.result;   // data URL
            preview.classList.remove('d-none');
          };
          reader.readAsDataURL(file);        // reads as base64
        }
      });
    });
});
```

Feature	Why It's Done This Way
DOMContentLoaded wrapper	Ensures the script runs only after all HTML elements are present in the DOM — prevents 'element not fo
alert-success/info only auto-dismiss	Danger and warning alerts stay permanently — they contain important error messages the user must rea
bootstrap.Alert.getOrCreateInstance	Bootstrap 5 API for controlling alert components. close() triggers the fade-out animation.
pathname active link	Base.html already uses request.endpoint for server-side active states. This JS handles edge cases wher
data-confirm attribute	Any HTML element can get a confirm dialog by adding data-confirm='Are you sure?'. No inline onclick ne

FileReader.readAsDataURL	Converts the selected image file into a base64 data URL string that can be set as img.src — shows previ

Section 8 — seed_data.py — Demo Data Seeding

188 lines that populate the database for demonstration

What seed_data.py inserts	Details
30 medicines	10 normal stock with valid expiry, 5 low stock (qty ≤ 10), 5 expiring soon (within 30 days), 4 ex
Medicine types	Tablet, Capsule, Syrup, Injection, Inhaler, Cream, Drops, Sachet — all 8 types for complete R
20 sales records	Spread across 9 days ago → today using timedelta offsets. Powers the 7-day sales chart on D
20 activity log entries	Login, add, update, delete, export, sale — for every user — populates the Logs page
Relative dates	All expiry and sale dates use date.today() ± timedelta(days=N) so data is always relevant rega
c.execute('DELETE FROM...')	Clears existing data first — running seed again gives clean slate (avoids duplicates)
conn.row_factory = sqlite3.Row	Needed to read back inserted data. Direct sqlite3 connection (not Flask's get_db).

Important: How to run seed_data.py

seed_data.py connects directly to the SQLite file — it does not go through Flask at all. It must be run from inside the pharmacy_app/ directory so the relative path 'instance/pharmacy.db' resolves correctly. Run ONCE after the first python app.py startup.

Section 9 — How Files Interact

Complete request lifecycle from browser to database and back

Request Flow: User clicks 'Add Medicine'



Step	File / Function	What Happens
1	Browser (any)	User fills the Add Medicine modal form and clicks 'Save Medicine'
2	medicines.html	HTML <form method='post' action='/add_medicine' enctype='multipart/form-data'> sends POST req
3	app.py: Flask router	Flask matches URL '/add_medicine' to the add_medicine() function
4	app.py: @login_required	Checks session['user_id'] exists. If not → redirect to /login. Else → continue.
5	app.py: @roles_required	Checks session['role'] in ['admin','pharmacist']. If not → redirect to /dashboard with error.
6	app.py: add_medicine()	Reads request.form fields + request.files. Calls secure_filename + file.save() for image.
7	app.py: query_db()	Calls get_db() → sqlite3.connect(DATABASE). Executes INSERT SQL with parameterised args.
8	instance/pharmacy.db	SQLite writes new row to medicines table. Returns lastrowid.
9	app.py: log_activity()	Inserts a row into activity_logs: user, action, details, IP, timestamp.
10	app.py: flash() + redirect()	Sets flash message in session. Returns HTTP 302 redirect to /medicines.
11	Browser	Browser follows redirect, makes GET /medicines request.
12	medicines.html	Route queries medicines table and renders the list page with the new entry visible.
13	base.html context_processor	inject_alerts() runs, queries expiring/low_stock counts, injects into navbar badge.

Request Flow: Dashboard Page Load

File	Role in Dashboard Load
app.py: dashboard()	Runs 10+ SQL queries: COUNT, SUM, 7-day loop. Calls get_alerts(). Serialises to json.dumps().
instance/pharmacy.db	Returns aggregated stats, recent records, sales chart data.
app.py: inject_alerts()	Context processor adds nav_expiring + nav_low_stock to template context (2 more SQL queries).
Jinja2 engine	Merges dashboard.html + base.html. Substitutes {{ variable }} placeholders. Evaluates {% if %} / {% for %
base.html	Renders navbar (with badge from context processor), sidebar (role-conditional links), flash messages are
dashboard.html	Renders stat cards, alert banners (if counts > 0), chart canvases, recent medicines table, activity log.
Bootstrap 5 (CDN)	Styles the grid, cards, badges, tables. Loaded in base.html <head>.
Chart.js (CDN)	{% block scripts %} in dashboard.html creates two Chart objects using the json.dumps() data.
main.js	Auto-dismiss success alerts, sidebar active link highlight, tooltip init — runs after DOM loads.

Request Flow: Barcode Scan (AJAX)

Step	Detail
User grants camera	Browser requests camera permission via Quagga.init(). Rear camera stream shown in #interactive div.
Quagga decodes barcode	QuaggaJS processes video frames in real-time. onDetected fires with result.codeResult.code string.
JS calls lookupBarcode()	JavaScript fetch('/api/barcode/8901030000001') — async GET request, no page reload
app.py: api_barcode_lookup()	Route decorated with @api_auth. Queries: SELECT * FROM medicines WHERE barcode=?
JSON response	Returns dict(med) as JSON if found. Returns {'error':'Not found'}, 404 if not.
JS showResult()	On success: builds HTML string with medicine details, sets resultBody.innerHTML. Shows result card.
No page navigation	Entire interaction happens without leaving the barcode page — pure JavaScript async flow

Section 10 — How the Project Was Built

Step-by-step development process from blank folder to running app

Phase 1: Project Setup

Step	Action	Why
1	Created directory pharmacy_app/ with subdirs: instance/, static/css/, static/uploads/, templates/	Flask convention for static files, templates, and uploaded images at app root
2	pip install flask werkzeug openpyxl → saved to requirements.txt	Only 3 dependencies needed — no database driver, ORM, or admin framework required
3	Wrote app.py skeleton: Flask(__name__), secret_key, DB_PATH, UPLOAD_FOLDER constants	Established all constants upfront so everything references same source of truth
4	Wrote init_db() with CREATE TABLE IF NOT EXISTS for all 4 tables	Schema-first: define data structure before writing any logic
5	Added default user seeding inside init_db(): admin, pharmacist, staff	Ensures app is immediately usable on first run without manual setup

Phase 2: Core Authentication

Step	What Was Built
6	login.html standalone page with gradient background, logo, form, demo credentials panel
7	/login route: GET renders form, POST validates with check_password_hash(), stores user in session
8	login_required decorator using @wraps to protect all subsequent routes
9	roles_required(*roles) decorator for admin-only and pharmacist-or-admin routes
10	log_activity() helper that inserts to activity_logs on every significant action

Phase 3: Base Layout & Navigation

Step	What Was Built
11	base.html with fixed navbar, collapsible sidebar, flash messages area, Bootstrap+Chart.js+main.js CDN links
12	@app.context_processor inject_alerts() so nav badge shows on all pages without touching every route
13	Role-conditional sidebar links: {% if session.role == 'admin' %} sections for Users/Logs
14	style.css: CSS variables, navbar gradient, sidebar sticky, stat cards, role badges, responsive breakpoints
15	main.js: auto-dismiss alerts, active link, confirm-delete, image preview — runs on every page

Phase 4: Medicines CRUD + Search + Export

Step	What Was Built
16	medicines.html table with pagination, search bar, colour-coded rows (red=expired, yellow=expiring/lowstock)
17	_medicine_form.html partial with all 16 fields — included in both Add and Edit modals
18	/add_medicine, /update_medicine/<id>, /delete_medicine/<id> routes with role protection and logging

19	Image upload: secure_filename, timestamp prefix, save to static/uploads/, store filename in DB
20	medicine_detail.html: full product view with breadcrumb, image, clinical info, status badges
21	/export_excel using openpyxl: green header row, colour-coded data rows, BytesIO in-memory serve

Phase 5: Alerts, Sales, Reports, Dashboard Charts

Step	What Was Built
22	get_alerts() shared helper — used by /alerts route AND context_processor for consistent counts
23	alerts.html: three sections (Expired/Expiring Soon/Low Stock) with colour-coded tables and summary cards
24	/sales route + add_sale with stock validation, atomic UPDATE medicines SET qty=qty-N, log_activity
25	/reports with GROUP BY medicine_type, Top 10 low stock, 30-day sales summary, json.dumps for Chart.js
26	Dashboard: 10 SQL queries, 7-day loop for chart data, json.dumps() → Chart.js line+donut charts

Phase 6: Advanced Features

Step	What Was Built
27	barcode.html: QuaggaJS init, camera stream, onDetected callback, async fetch to /api/barcode/, scan history
28	REST API: api_auth decorator, 7 JSON endpoints (GET/POST/PUT/DELETE) for external integrations
29	User management: /users, /add_user with duplicate check, /delete_user with self-deletion guard
30	Profile page: edit name+email, change password with current-password verification
31	Activity logs page: paginated (20/page), colour-coded action badges, IP address display
32	seed_data.py: 30 medicines with all status types, 20 sales across 9 days, 20 log entries — relative dates

Design Decisions & Patterns Used

Pattern	Where Used	Why
PRG (Post/Redirect/Get)	All form submissions	Prevents duplicate form submission on browser refresh — flash message survives re
Template inheritance	All pages extend base.html	Write navbar/sidebar/scripts once, reuse everywhere. Change base.html = change e
Parameterised queries	Every SQL in query_db()	Prevents SQL injection attacks — args=() passes values as parameters, never string
Shared helper functions	get_alerts(), query_db(), log_activity()	DRY (Don't Repeat Yourself) — one place to fix bugs, one place to change logic
Context processor	inject_alerts()	Separates cross-cutting concern (alert counts) from individual route logic — runs tra
Soft foreign keys	sales.medicine_name	Denormalized name stored in sales — preserves sale record even if medicine delete
Flask g object	get_db()	One DB connection per request, shared across query_db() calls. Prevents connectio
Atomic SQL updates	product_qty=product_qty-N	Race condition safe — SQL does the arithmetic server-side, not Python client-side
BytesIO for Excel	export_excel()	No temp files on disk — Excel generated in RAM, streamed to browser, garbage coll
json.dumps for charts	dashboard, reports routes	Bridge between Python data and JavaScript arrays — the |safe Jinja filter prevents H